

ZiaLabs AI — Project Documentation & Commands Guide

Yeh document un saari cheezon ka detailed summary hai jo humne is ZiaLabs AI Research Co-Pilot project mein develop ki hain.

1. Project Setup Commands

Project ko run aur manage karne ke liye sabse important commands:

- Install Dependencies:** `npm install`
(Ye saari required libraries install karta hai jaise express, bcryptjs, jsonwebtoken, @google/generative-ai, xml2js, aur sql.js)
- Start Server (Production/Normal mode):** `npm start`
- Start Server (Development mode with auto-reload):** `npm run dev`
- Stop Server:** Terminal mein `Ctrl + C` press karein.

2. Environment Variables (.env)

Application ko chalane ke liye .env file mein ye configurations set ki gayi hain:

- `PORT=3000` — Backend server ka port.
- `JWT_SECRET` — User passwords aur sessions ko secure rakhne ke liye secret key.
- `GEMINI_API_KEY` — Google Gemini AI agent ko chalane ke liye API key.

3. Backend Architecture & Functions (Node.js + Express)

Humne project ko ek clean MVC (Model-View-Controller) structure mein banaya hai.

A. Core Services (server/services/)

- AI Agent .js:** Yeh core AI brain hai. Isme Gemini API (gemini-flash-latest) integrate ki gayi hai jo user ke questions ka answer deti hai. Har user ki chat history maintain hoti hai.
- ArxivService .js:** Yeh function ArXiv ke open API (<http://export.arxiv.org/api/query>) ko call karke research papers fetch karta hai. Isko kisi API key ki zarurat nahi hoti.
- SemanticScholarService .js:** Yeh Semantic Scholar ki API use karke papers search karta hai aur unki citations count deta hai.
- AuthService .js:** User registration, login aur JWT token generate karne ka logic yahan likha gaya hai.

B. Database Models (server/models/)

Humne sql.js (Pure JavaScript SQLite) use kiya hai taaki Windows par C++ errors na aayein.

- User .js:** User create karna, password verify karna, aur API calls ki limit track karna.
- Paper .js:** Users dwara "Save" kiye gaye papers ko database mein permanently store karna.
- ChatMessage .js:** AI aur user ke beech ki baaton ko save karna taaki conversation context bana rahe.
- SearchHistory .js:** User ne kya search kiya, uska record rakhna.

C. API Routes (server/routes/)

- Auth API (/api/auth):** /register, /login, aur /me endpoints authentication ke liye.
- Search API (/api/search):** /arxiv aur /semantic-scholar APIs jo frontend se search query lekar papers return karti hain.
- Papers API (/api/papers):** Saved library papers ko Fetch (GET /), Save (POST /), aur Delete (DELETE /:id) karne ke functions.
- Chat API (/api/chat):** AI se baat karne ke liye /message, history load karne ke liye /history, aur chat clear karne ke liye /clear.

4. Frontend Application (public/)

Frontend bina kisi framework (React/Angular) ke Pure Vanilla JavaScript mein likha gaya hai taaki fast load ho.

- index.html:** Single Page Application (SPA) structure. Isme Landing Page, Sign In/Up forms aur main Dashboard ek hi file mein hain, jo JavaScript se hide/show hote hain.
- css/style.css:** Modern Dark UI with glassmorphism effects, variables (:root) color scheme, aur mobile responsive design.
- js/app.js:** Isme Object-Oriented JS Classes banayi gayi hain:
 - ApiClient:** Backend se data lane ke liye fetch wrapper.
 - Auth:** Login/Logout aur token management.
 - Dashboard:** Sidebar navigation, views switch karna, aur papers show karna.
 - Search:** Search bar handle karna aur ArXiv API ko call karke UI update karna.
 - Library:** Saved papers ko render karna.
 - Chat:** AI chatbot UI handle karna, message append karna aur auto-scroll karna.

5. Security & Fallback Features

- JWT Tokens:** API endpoints sirf logged-in users ko hi data dete hain. authMiddleware har request ko verify karta hai.
- Graceful Degradation:** Agar .env mein Gemini API key galat ho ya na ho, toh server crash nahi hota, balki AI Agent "Fallback mode" mein chala jata hai aur user ko properly guide karta hai.
- Password Hashing:** bcryptjs ka use kiya gaya hai taaki password text form mein save na hon.